CUDA_Experiment_8\vector_multiply.cu

```
#include <stdio.h>
#include <cuda_runtime.h>

#define CUDA_CHECK(call) do { \
    cudaError_t err__ = (call); \
    if (err__ != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__, cudaGetErrorString(err__)); \
        return 1; \
    } \
} while (0)

__global__ void vectorMultiply(const int *A, const int *B, int *C, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) C[i] = A[i] * B[i];
}

int main() {
    const int N = 5;
    int h_A[N] = {10, 20, 30, 40, 50};
    int h_B[N] = {1, 2, 3, 4, 5};
    int h_C[N] = {0};
    int *d_A = NULL, *d_B = NULL, *d_C = NULL;

    CUDA_CHECK(cudaMalloc((void**)&d_A, N * sizeof(int)));
    CUDA_CHECK(cudaMalloc((void**)&d_B, N * sizeof(int)));
    CUDA_CHECK(cudaMalloc((void**)&d_C, N * sizeof(int)));
    CUDA_CHECK(cudaMemcpy(d_A, h_A, N * sizeof(int), cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_B, h_B, N * sizeof(int), cudaMemcpyHostToDevice));

    vectorMultiply<<<1, N>>>(d_A, d_B, d_C, N);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(h_C, d_C, N * sizeof(int), cudaMemcpyDeviceToHost));

    printf("Vector Multiplication Result:\n");
    for (int i = 0; i < N; ++i)
        printf("%d * %d = %d\n", h_A[i], h_B[i], h_C[i]);

    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    return 0;
}

```